LAPORAN PRAKTIKUM

TUGAS – Praktik 7

(TEKNOLOGI IOT II)

Disusun Oleh :

Nama : Feni Afelia
Kelas : 3A
NPM : 25782011

Dosen :

1. Dr. Ir. Septafiansyah Dwi Putra
2. Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG 2026

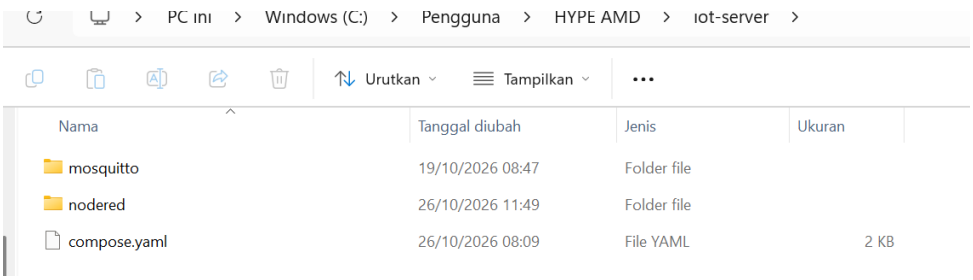
➢ HASIL PELAKSANAAN PRAKTIKUM

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: iot-server (misalnya pada direktori C:/iot-server atau C:/Users/NamaAnda/iot-server).

2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:

```
C:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───nodered
    ├───lib
    │   └───flows
    └───node_modules
```



3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama mosquitto.conf, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous true

persistence true
persistence_location /mosquitto/data/

log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

4. Kembali ke folder induk iot-server, buat berkas baru bernama docker-compose.yml, lalu salin konfigurasi layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

5. Jalankan Terminal (Linux/macOS) atau PowerShell / Command Prompt (Windows), arahkan direktori aktif ke dalam folder iot-server:

```
C:\Users\user>cd iot-server
```

6. Eksekusi perintah berikut untuk mengunduh (pull) image dan menjalankan kontainer di latar belakang (detached mode):

```
C:\Users\HYPE AMD\iot-server>docker compose up -d
[+] up 2/2
 ✔Container mqtt-broker Started
 ✔Container node-red    Started
```

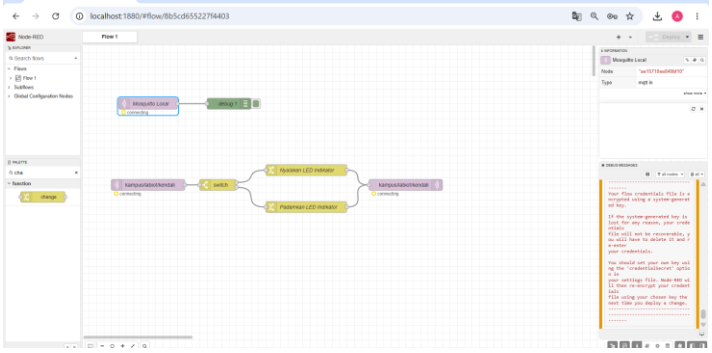
7. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

```
C:\Users\HYPE AMD\iot-server>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED         STATUS                            PORTS                                       NAMES
db671a732d51   nodered/node-red:latest  "./entrypoint.sh"        4 minutes ago   Up 16 seconds (health: starting)  0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp  node-red
b14bfd3f7e62   eclipse-mosquitto:latest "/docker-entrypoint…"    4 minutes ago   Up 17 seconds                     1883/tcp, 9001/tcp                          mqtt-broker
```

8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

```
C:\Users\HYPE AMD\iot-server>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED         STATUS                            PORTS                                       NAMES
db671a732d51   nodered/node-red:latest  "./entrypoint.sh"        4 minutes ago   Up 16 seconds (health: starting)  0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp  node-red
b14bfd3f7e62   eclipse-mosquitto:latest "/docker-entrypoint…"    4 minutes ago   Up 17 seconds                     1883/tcp, 9001/tcp                          mqtt-broker
```

9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: http://localhost:1880. Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Rangkaian perangkat keras:



Langkah Pengerjaan:
1. Cari tahu Alamat IPv4 Lokal laptop Anda yang terhubung ke jaringan Wi-Fi:

```
PS C:\Users\HYPE AMD> ipconfig

Windows IP Configuration


Ethernet adapter vEthernet (Default Switch):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::3e11:87e6:fcc6:6d12%23
   IPv4 Address. . . . . . . . . . . : 172.31.112.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

Ethernet adapter vEthernet (WSL (Hyper-V firewall)):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8ad2:b2da:b3b5:ba5c%56
   IPv4 Address. . . . . . . . . . . : 172.25.96.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

Ethernet adapter Ethernet 3:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::c217:a68b:fe65:1a86%7
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::e70a:d05d:516a:d1c1%20
   IPv4 Address. . . . . . . . . . . : 172.20.10.3
   Subnet Mask . . . . . . . . . . . : 255.255.255.240
   Default Gateway . . . . . . . . . : 172.20.10.1

Ethernet adapter Bluetooth Network Connection:

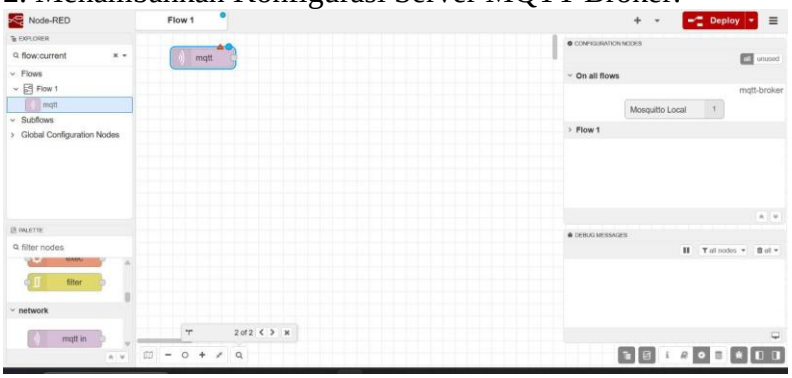
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

2. Kode dan Output:

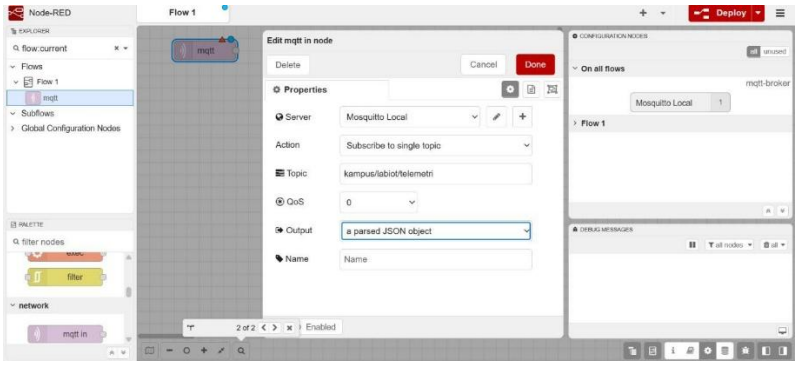


7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

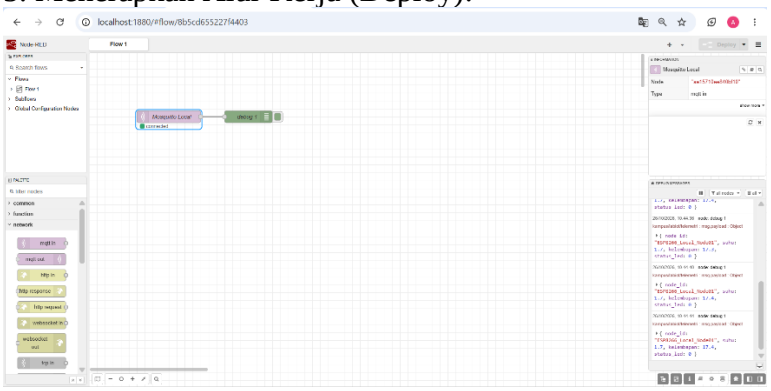
1. Buka peramban web dan akses antarmuka Node-RED di http://localhost:1880.
2. Menambahkan Konfigurasi Server MQTT Broker:



3. Mengonfigurasi Parameter Topik:



4. Menambahkan Node Pemantau (Debug Node)
5. Menerapkan Alur Kerja (Deploy):

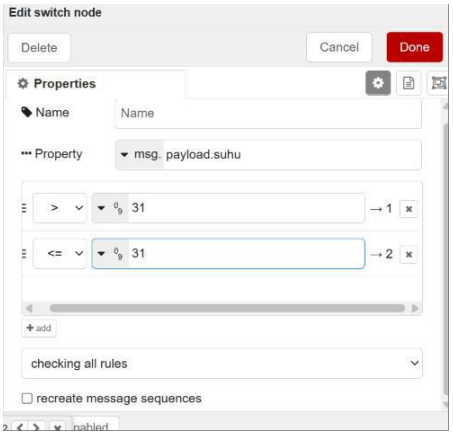


6. Menganalisis Aliran Data:

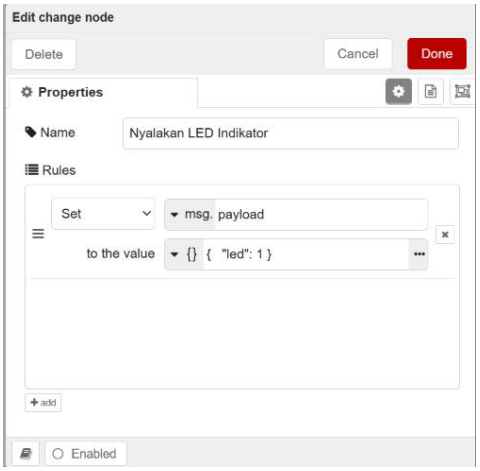


7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

1. Membuat Node Percabangan Logika (switch):



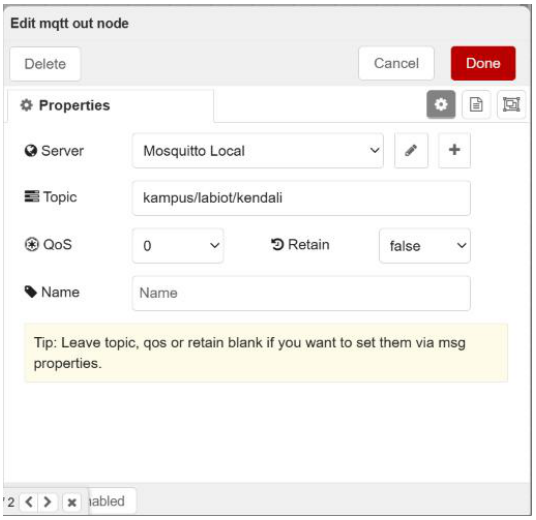
2. Membuat Node Transformasi Muatan Panas (change)

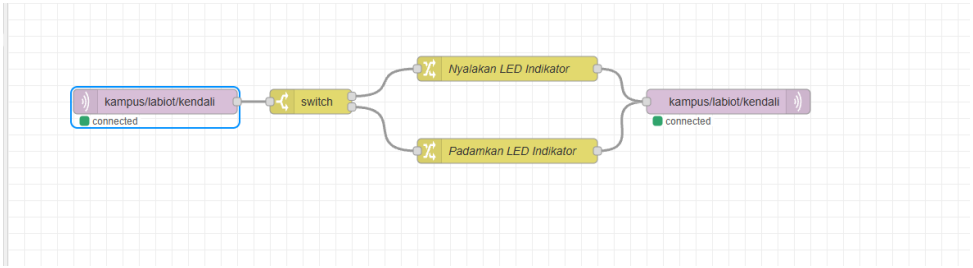


3. Membuat Node Transformasi Muatan Normal (change):

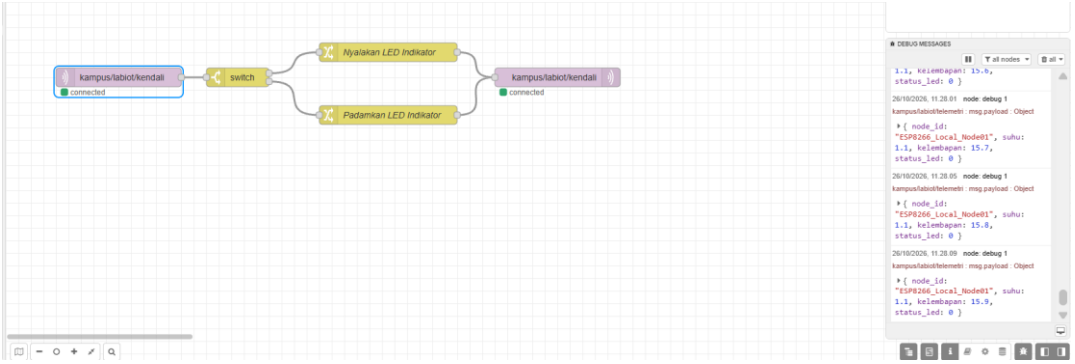


4. Menerbitkan Instruksi Balik ke Broker (mqtt out):





5. Menerapkan Perubahan (Deploy)
6. Pengujian Fisik Sistem Otomasi Tertutup:



- **HASIL JAWABAN LATIHAN (LATIHAN 1 - 4)**

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):

Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (Connection Refused)? Jelaskan bagaimana mekanisme Docker Embedded DNS Server dan Bridge Network bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jawab:

1. Mengapa localhost menyebabkan kegagalan koneksi?

localhost merujuk pada perangkat atau lingkungan tempat aplikasi sedang berjalan. Karena Node-RED berjalan di dalam kontainer Docker, localhost akan menunjuk ke kontainer Node-RED itu sendiri, bukan ke kontainer Mosquitto. Akibatnya, ketika Node-RED mencoba terhubung ke broker MQTT melalui localhost:1883, koneksi dapat gagal dengan pesan Connection Refused karena tidak ada broker MQTT yang mendengarkan pada port tersebut di dalam kontainer Node-RED.

2. Bagaimana Docker Embedded DNS Server dan Bridge Network bekerja?

Docker menyediakan mekanisme DNS internal yang memungkinkan kontainer menemukan layanan lain berdasarkan nama service. Pada konfigurasi docker-compose.yml, service broker diberi nama mqtt, sedangkan service Node-RED diberi nama nodered. Kedua service tersebut tergabung dalam jaringan Docker Compose yang sama. Docker menyediakan jaringan bridge untuk komunikasi antarkontainer dan DNS internal untuk menerjemahkan nama service mqtt menjadi alamat IP internal kontainer broker.
Dengan demikian, Node-RED dapat menggunakan konfigurasi berikut:

- Server: mqtt
- Port: 1883

Node-RED tidak perlu mengetahui alamat IP internal broker secara langsung. Docker menangani penerjemahan nama service dan komunikasi jaringan di antara kedua kontainer.

Kesimpulan: Penggunaan nama service mqtt membuat konfigurasi lebih sederhana dan tidak bergantung pada alamat IP internal kontainer yang dapat berubah ketika kontainer dibuat ulang.

Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):

Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain broadcast / subnet yang sama?

Jawab:

1. Mengapa NodeMCU tidak dapat memublikasikan data?

NodeMCU memperoleh alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15. Dengan asumsi keduanya menggunakan subnet mask umum 255.255.255.0 (/24), kedua perangkat berada pada subnet yang berbeda.

Karena berbeda subnet, komunikasi langsung pada jaringan lokal tidak dapat dilakukan tanpa mekanisme routing yang sesuai. Jika jaringan Wi-Fi tidak menyediakan rute menuju laptop, NodeMCU tidak dapat menjangkau broker Mosquitto yang berjalan pada laptop. Selain perbedaan subnet, koneksi juga dapat terhambat apabila firewall laptop memblokir port MQTT 1883 atau broker hanya mendengarkan pada alamat loopback.

2. Langkah teknis agar kedua perangkat berada dalam subnet yang sama

Langkah yang dapat dilakukan adalah:
1. Hubungkan laptop dan NodeMCU ke jaringan Wi-Fi atau hotspot yang sama.
2. Pastikan keduanya mendapatkan alamat IP dalam subnet yang sama dari DHCP jaringan tersebut.
3. Periksa alamat IP laptop melalui Command Prompt menggunakan perintah: ipconfig
4. Gunakan alamat IPv4 laptop sebagai alamat broker MQTT dalam kode ESP8266, misalnya:

```
const char* mqttServer = "192.168.43.15";
const int mqttPort = 1883;
```

5. Pastikan Mosquitto mendengarkan pada antarmuka jaringan yang dapat dijangkau ESP8266 dan port 1883 diizinkan oleh firewall laptop.
6. Uji koneksi menggunakan MQTT Explorer data.

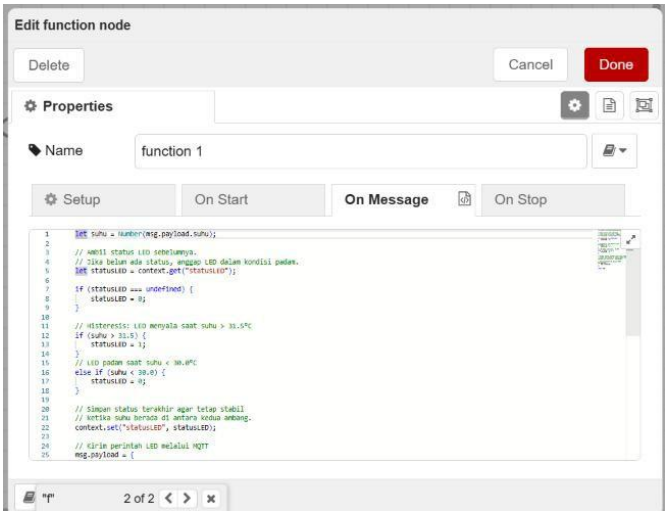
Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):

Pada sistem kendali dua posisi (Bang-Bang), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0◦ C (misal: 31.05◦ C→30.95◦ C→31.02◦ C), lampu LED indikator akan berkedip-kedip cepat (flickering / hunting) yang mengganggu stabilitas visual sistem. Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:

LED baru menyala jika suhu >31.5◦ C (msg.payload = {"led": 1}).

LED baru padam jika suhu mendingin hingga <30.0◦ C (msg.payload = {"led": 0}).
Tuliskan kode JavaScript yang Anda masukkan ke dalam Function Node tersebut!

Jawab:
Tambahkan satu Function Node setelah mqtt in, lalu masukkan kode berikut:



Histeresis mengurangi pergantian status LED yang terlalu sering akibat fluktuasi suhu kecil. Dua ambang batas membuat sistem lebih stabil dibandingkan penggunaan satu ambang batas saja.

Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):

Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
Jelaskan perbedaan mendasar antara Bind Mount dan Named Volume pada Docker!

Jawab:
Jika pemetaan ./nodered:/data dihapus, data Node-RED seperti flow, pengaturan, dan data pengguna yang disimpan di /data hanya berada pada writable layer kontainer. Ketika kontainer dihapus menggunakan docker compose down lalu dibuat ulang dengan docker compose up -d, writable layer kontainer lama ikut dihapus. Akibatnya, flow yang belum disimpan di lokasi persisten dapat hilang dan Node-RED mungkin kembali ke kondisi awal. Hal serupa berlaku pada data Mosquitto. Jika pemetaan ./mosquitto/data:/mosquitto/data dihapus, data yang ditulis di direktori tersebut tidak lagi tersimpan pada folder host. Data dapat hilang saat kontainer dihapus dan dibuat ulang. docker compose down biasanya menghapus kontainer dan jaringan Compose, tetapi tidak menghapus direktori bind mount pada laptop. Oleh karena itu, jika bind mount tetap digunakan, data yang tersimpan di folder host tetap ada meskipun kontainer dibuat ulang.

- **Perbedaan Bind Mount dan Named Volume**

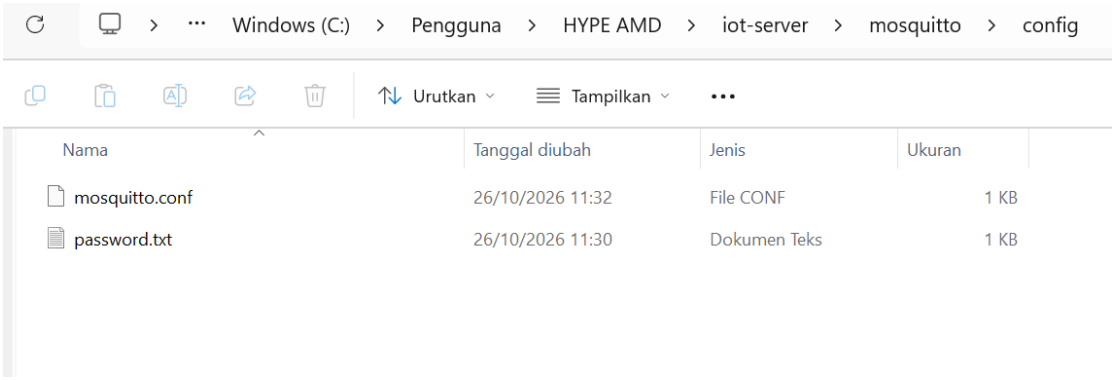
Aspek	Bind	Named Volume
Lokasi penyimpanan	Lokasi direktori atau file yang ditentukan pada host	Dikelola Docker pada lokasi penyimpanan volume
Cara penulisan	Contoh: ./nodered:/data	Contoh: nodered_data:/data
Akses langsung	Mudah dibuka dan diedit melalui folder host	Umumnya dikelola melalui perintah Docker
Portabilitas	Bergantung pada struktur direktori host	Lebih mudah dikelola sebagai objek volume Docker
Kegunaan umum	Pengembangan, konfigurasi, dan file yang perlu diedit langsung	Persistensi data aplikasi dan penyimpanan yang dikelola Docker

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
PS C:\Users\HYPE AMD> docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/
```



kini terdapat berkas baru bernama password.txt yang berisi baris hash kredensial akun admin

2 Memperbarui Konfigurasi Kebijakan Mosquitto:

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous true

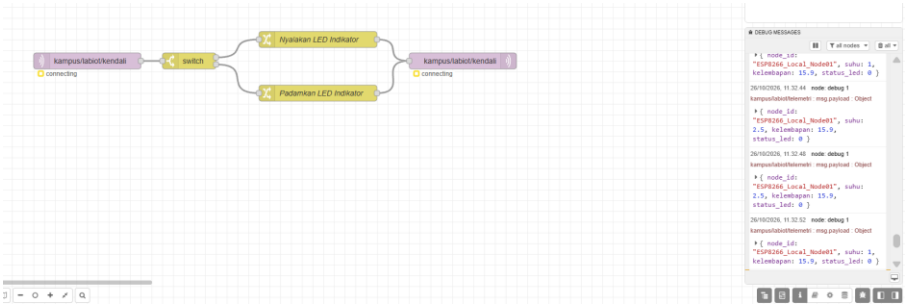
persistence true
persistence_location /mosquitto/data/

log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Memuat Ulang Layanan Broker:

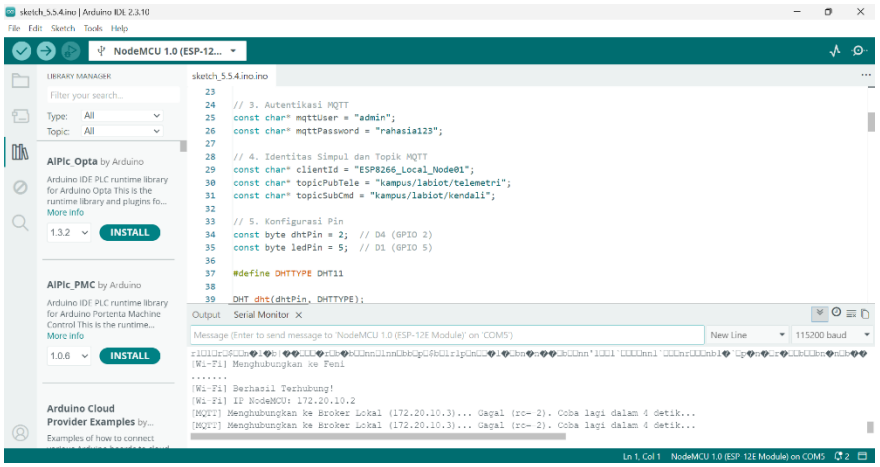
```
PS C:\Users\HYPE AMD> docker restart mqtt-broker
mqtt-broker
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

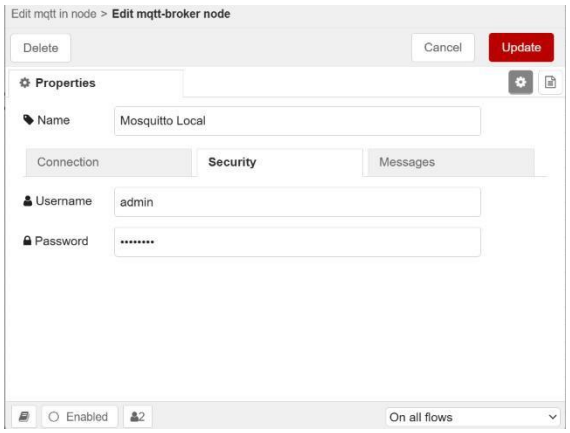


Setelah password diaktifkan, koneksi Node-RED yang masih tanpa kredensial akan terputus

Lalu di monitor arduino uno nya akan seperti ini:



5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):



Buka tab Security Masukkan Username: admin dan Password: rahasia123. lalu klik update dan deploy.

Hasil nya node red kembali terhubung dan monitor di arduino uno kembali berjalan

